Force Reconstruction Algorithm (Version 1.4)

Input:

- $n_{sensors}$ (number of sensing channels)
- $N_W = 1000$ (integration window length)
- $Thr_{samples} = 1500$ (noise learning samples)
- $n_\sigma = 10.0$ (threshold scale factor)
- $press_{confirm} = 5$ (trigger confirmation samples)
- $fifo_{len} = 50$ (release slope FIFO length)
- $avg_{multiplier} = 0.15$ (release slope multiplier)
- $\alpha = 0.1$ (EMA smoothing factor)
- $n_{offset} = 50$ (adaptive offset window)
- $samples_{after\_release} = 100$ (guard samples)
- $signal2noise = 10.0$ (minimum SNR)
- $min_{press\_sensors} = 2$ (global press quorum)
- $release_{ratio} = 0.5$ (global release ratio)

Initialization

$noise\_counter \leftarrow 0$
$noise\_buffer[Thr_{samples}, n_{sensors}] \leftarrow 0$
$adaptive\_offset[n_{sensors}] \leftarrow 0$
$offset\_buffer[n_{offset}, n_{sensors}] \leftarrow 0$
$pre\_buffer[press_{confirm} + 5, n_{sensors}] \leftarrow 0$
$press\_buffer[N_W, n_{sensors}] \leftarrow 0$
$fifo\_buffer[fifo_{len}, n_{sensors}] \leftarrow 0$
$x_{prev}[n_{sensors}] \leftarrow 0$
for $ns = 1$ to $n_{sensors}$ **do**
    $integral[ns] \leftarrow 0$
    $counter[ns] \leftarrow 0$
    $confirm[ns] \leftarrow 0$
    $fifo\_idx[ns] \leftarrow 0$
    $triggered[ns] \leftarrow False$
    $validated[ns] \leftarrow False$
    $armed[ns] \leftarrow True$
    $press\_sign[ns] \leftarrow +1$
    $max_{post}[ns] \leftarrow 0$
    $second\_cross[ns] \leftarrow False$
    $idx_{second\_cross}[ns] \leftarrow 0$
    $press\_integral[ns] \leftarrow 0$
    $average_{touch}[ns] \leftarrow 0$
    $guard\_counter[ns] \leftarrow 0$
    $previous\_event\_polarity[ns] \leftarrow 0$
$global\_press \leftarrow False$
$press\_mask[1..n_{sensors}] \leftarrow False$
$release\_mask[1..n_{sensors}] \leftarrow False$

1. Noise Learning Phase

for $k = 1$ to $Thr_{samples}$ **do**

```
    Store x_raw[k, 1..n_sensors] into noise_buffer
σ[ns] ← std(noise_buffer[:, ns])
thr_press[ns] ← press_σ · σ[ns]
max_noise[ns] ← 99^th%(|noise_buffer[:, ns] − median|)
Initialize offset_buffer and adaptive_offset
```

2. Online Processing Loop

```
for each new sample x_raw[1..n_sensors] do
    for ns = 1 to n_sensors do
        x_smooth ← α x_raw[ns] + (1 − α) x_prev[ns]
        x_prev[ns] ← x_smooth
        x ← x_smooth − adaptive_offset[ns]
        if guard_counter[ns] > 0 then
            guard_counter[ns] ← guard_counter[ns] − 1
            Output 0
            continue
        if |x| < 10 · thr_press[ns]/press_σ then
            Update rolling mean of offset_buffer[:, ns]
            Update adaptive_offset[ns]
        Shift pre_buffer[:, ns], append x
        2.1 Trigger Detection
        if not triggered[ns] then
            if armed[ns] and |x| > thr_press[ns] then
                confirm[ns] ← confirm[ns] + 1
                if confirm[ns] ≥ press_confirm then
                    triggered[ns] ← True
                    armed[ns] ← False
                    confirm[ns] ← 0
                    second_cross[ns] ← False
                    idx_second_cross[ns] ← 0
                    press_sign[ns] ← sign(∑ pre_buffer[:, ns])
                    if press_sign[ns] = 0 then
                        press_sign[ns] ← sign(x)
                    integral[ns] ← ∑ press_sign[ns] · pre_buffer[:, ns]
                    press_buffer[0 : pre, ns] ← cumsum(press_sign[ns] · pre_buffer[:, ns])
                    counter[ns] ← pre
            else
                confirm[ns] ← 0
            Output 0
            continue
        2.2 Integration
        integral[ns] ← integral[ns] + press_sign[ns] · x
        if integral[ns] < 0 then
            Reset local state, enable guard
            Output 0
            continue
        max_post[ns] ← max(max_post[ns], |x|)
        if counter[ns] < N_W then
            press_buffer[counter[ns], ns] ← integral[ns]
            counter[ns] ← counter[ns] + 1
            Output 0
            continue
        Output integral[ns]
        2.3 Second Threshold Crossing
        if |x| < thr_press[ns] and not second_cross[ns] then
            idx_second_cross[ns] ← counter[ns]
```

$press\_integral[ns] \leftarrow integral[ns]$
$second\_cross[ns] \leftarrow True$

2.4 Window Validation

if not $validated[ns]$ **then**
  $idx \leftarrow \arg\max press\_buffer[0 : idx_{second\_cross}, ns]$
  $avg \leftarrow \dfrac{-press\_buffer[0, ns] + press\_buffer[idx, ns]}{idx + 1}$
  if $max_{post}[ns] < signal2noise \cdot max_{noise}[ns]$ **then**
    Reset local state (no guard)
    continue
  $average_{touch}[ns] \leftarrow avg$
  $validated[ns] \leftarrow True$
  $press\_mask[ns] \leftarrow True$

2.5 Global Press Consensus

if not $global\_press$ **then**
  if $\sum press\_mask \geq min_{press\_sensors}$ **then**
    $global\_press \leftarrow True$

2.6 Release Detection (FIFO Slope)

if $validated[ns]$ **and** $global\_press$ **then**
  Update $fifo\_buffer[:, ns]$
  if FIFO full **then**
    $slope \leftarrow \dfrac{fifo[-1] - fifo[0]}{fifo_{len}}$
    if $slope < -slope_{mult}|average_{touch}[ns]|$ **or** $integral[ns] > 1.5\, press\_integral[ns]$ **or** $integral[ns] < 0.5\, press\_integral[ns]$ **then**
      $release\_mask[ns] \leftarrow True$

2.7 Global Release

if $\sum release\_mask / \sum press\_mask \geq release_{ratio}$ **then**
  Reset all sensors with guard enabled
  $press\_mask[:] \leftarrow False$
  $release\_mask[:] \leftarrow False$
  $global\_press \leftarrow False$